

Database Lab — Chat Transcript

September 12–13, 2026

User

Task Description

1. Select six recent popular movies (or television episodes or books or songs or ...)
2. Ask at least five people to rate each movie they have seen on a 1–5 scale.
3. Store the collected ratings in a SQL database of your choice.
4. Load the data from SQL into R as a dataframe.

These are the tasks for my database lab. When I surveyed people some of them did not watch every movie. So I was thinking of having 3 tables: Users(user_id, name), movies(movie_id, title) and ratings(user_id, movie_id, rating). Part of the lab is having a way to deal with missing values, I figured that I can put NULL in for those values. Is that a good way to deal with missing values, or is there a better method.

Assistant

Yes — your three-table design is the better approach, but I would not put NULL into the ratings table for movies someone didn't watch.

The important distinction is between “no rating exists” and “a rating exists but its value is missing.”

For a person who didn't watch a movie, simply don't create a row for that person/movie combination. No row communicates that the person did not rate the movie. A row with NULL would instead suggest that a rating record exists but the rating itself is unknown/missing.

This can still give you a legitimate missing-data problem to discuss in R. If you create the complete set of person/movie combinations, missing ratings can be represented as NA in R. SQL commonly uses NULL, while R uses NA.

The three-table design is also a standard many-to-many structure: Users and Movies are connected through the Ratings junction table. A primary key on (user_id, movie_id) can ensure that each person has only one rating per movie.

User

I provided my SQL INSERT statements for six users, six movies, and their ratings. Everything ran smoothly up until inserting into the ratings section. I received:

```
ERROR: duplicate key value violates unique constraint "ratings_pkey"  
Key (user_id, movie_id)=(1, 3) already exists.
```

I asked why the error was occurring.

Assistant

The error is happening because of the primary key constraint on ratings, likely defined as PRIMARY KEY (user_id, movie_id). This means the combination of user_id and movie_id must be unique: a particular user can only have one rating for a particular movie.

The error identifies the combination causing the problem: (user_id, movie_id) = (1, 3). Your users were inserted in this order: 1 = Elvy, 2 = Chrissie, 3 = Kaitlin, 4 = Renan, 5 = Jackson, 6 = Isaiah.

Earlier, Elvy was inserted with (1, 3, 5), meaning user 1 rated movie 3 a 5. Later, in the section labeled Chrissie's ratings, you inserted (1, 3, 4). Because 1 is Elvy's user_id, PostgreSQL interprets this as Elvy rating movie 3 again. That violates the primary-key rule because (1, 3) already exists.

There is another similar issue in Chrissie's section with (1, 6, 4): user 1 is also Elvy, and Elvy already has a rating for movie 6. The database design is working correctly; the issue is that the wrong user_id was entered in those INSERT statements.